

LGTM 프로젝트 결과보고서

[프로젝트 기본 정보]

- 프로젝트명: LGTM Observability Stack
- 수행 기간: 2026. 06. 08. ~ 2026. 06. 18. (약 2주 수행)
- 배포 환경: Cloud VM (IXcloud, Ubuntu 22.04 LTS, Docker Compose 기반)
- 외부 접속 엔드포인트: <http://1.201.177.179:3000/> (Grafana)
- 산출물(Git): https://github.com/kinx12399/LGTM_Observability_stack
- 내부 네트워크: Docker bridge network `lgtm-net` 기반 격리 통신

1. 프로젝트 개요

1.1 프로젝트 목적

- **통합 관찰성(Observability) 환경 구축:** 단일 Cloud VM 환경 내에서 LGTM(Loki, Grafana, Tempo, Mimir) 스택을 연동하여, 시스템 및 인프라의 로그, 메트릭, 트레이스를 한곳에서 모니터링할 수 있는 고도화된 통합 관찰성 플랫폼을 구현합니다.
- **오픈소스 기반 모니터링 역량 강화:** Promtail, Node Exporter 등 다양한 에이전트를 활용해 데이터를 수집하고, 효율적인 오픈소스 인프라 운영 및 모니터링 아키텍처 설계 능력을 확보하는 것을 목적으로 합니다.

1.2 프로젝트 범위

- **인프라 및 시스템 환경:** Cloud VM (IXcloud, Ubuntu 22.04 LTS) 상에 Docker Compose 기반으로 가상화 환경을 격리 및 구축 (`lgtm-net` 브릿지 네트워크 활용)
- **LGTM 스택 및 에이전트 구축:**
 - **Grafana:** 통합 모니터링 시각화 및 대시보드 구축, 알람(Alert) 규칙 및 Contact Point 설정
 - **Loki & Promtail:** 시스템 및 Docker 로그 실시간 수집 및 중앙 저장소 구축
 - **Mimir, Prometheus & Node Exporter:** 15초 주기의 하드웨어/자원 메트릭 수집 및 시계열 데이터 저장
 - **Tempo:** 분산 트레이싱 기반 마련 및 gRPC(포트 4317) 연동
- **안정성 및 검증:** 각 컴포넌트의 Health Check, 스트레스 테스트(부하 테스트)를 통한 디스크 및 시스템 알람 기능 검증, 장애 시나리오 대응 및 RESOLVED 정책 수립

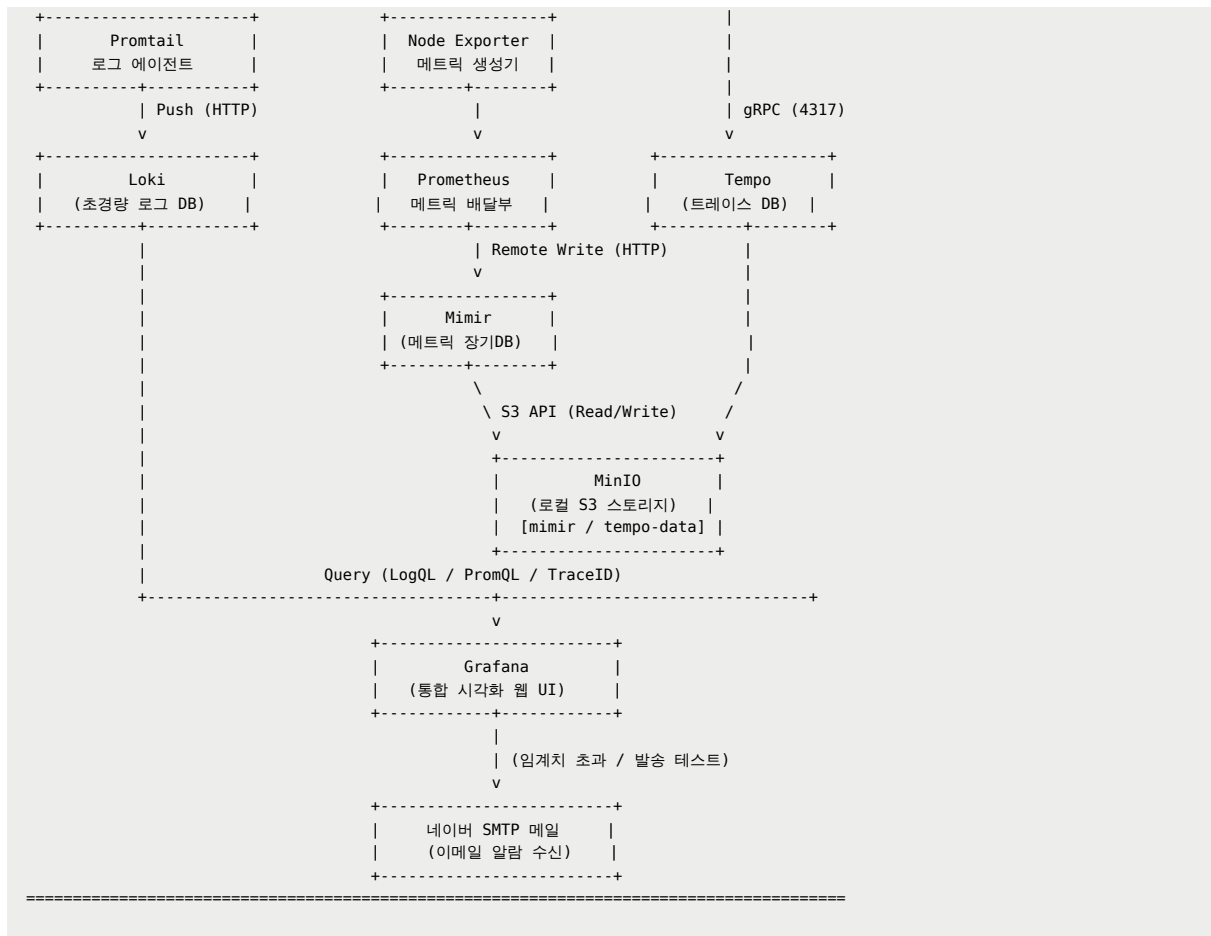
1.3 수행 기간

- 총 수행 기간: 2026. 06. 08. ~ 2026. 06. 18. (약 2주간 수행)

1. 아키텍처 설명

1.1 전체 텍스트 아키텍처 다이어그램





1.2 컴포넌트 간 데이터 흐름(Logs/Metrics/Traces)

1.2.1 로그 파이프라인: `/var/log/*log` → Promtail → Loki

- **로그 소스:** VM의 `/var/log/*log` (syslog/auth.log/kern.log 등)
- **수집 방식:** Promtail이 파일 tailing 방식으로 **실시간 감시(scratch/tail)** 수행
- **전송 방식:** Promtail → Loki로 **HTTP Push** (`/loki/api/v1/push`)
- **가치:** 서비스 장애 시 메트릭과 함께 로그 이벤트를 빠르게 특정 가능

1.2.2 메트릭 파이프라인: Node Exporter → Prometheus → Mimir

- **메트릭 소스:** Node Exporter가 CPU/Memory/Disk/Network 등 호스트 메트릭을 `/metrics` 로 노출(9100)
- **수집 방식:** Prometheus가 `scrape_interval: 15s` 로 **Pull/Scrape**
- **저장 방식:** Prometheus가 Mimir로 **Remote Write**하여 장기 저장(9009)
- **가치:** 단일 VM이라도 시계열 장기 보관이 가능하여 “용량 추세/성능 추세” 기반 운영 판단이 가능

1.2.3 트레이스 파이프라인: OTLP gRPC(4317) → Tempo → MinIO

- **수집 방식:** Tempo가 OTLP gRPC endpoint(4317)를 열고 애플리케이션/에이전트가 보낸 트레이스를 수신
- **저장 방식:** Tempo는 WAL(write-ahead log)을 로컬 볼륨에 기록하고, 최종적으로 S3 호환 스토리지(MinIO)로 trace block을 저장
- **가치:** MSA 전환 시 서비스 간 호출 체인을 TraceID로 추적하여 병목/오류 위치를 빠르게 식별 가능

1.3 포트 구성 및 보안 권한(외부 노출 최소화)

요구사항서의 보안 권고에 따라 **Grafana(3000)만 외부 오픈**, 나머지 컴포넌트는 **Security Group**에서 **외부 접근을 차단**하고 Docker 내부 네트워크 **lgtn-net**에서만 통신하도록 구성한다.

컴포넌트	포트(컨테이너)	통신 방향	외부 오픈	비고
Grafana	3000	외부 브라우저 → Grafana	허용	대시보드/알림/데이터소스 통합
Loki	3100	Promtail → Loki	차단	LogQL 대상 로그 저장
Promtail	9080(관리 API)	내부(필요시)	차단	로그 수집 에이전트
Prometheus	9090	Prometheus → Node Exporter / Mimir	차단	Scrape + Remote Write 수행
Node Exporter	9100	Prometheus → Node Exporter	차단	호스트 메트릭 노출
Mimir	9009	Prometheus → Mimir / Grafana → Mimir	차단	장기 메트릭 저장(모놀리식)
Tempo	3200(HTTP), 4317(gRPC)	App/Agent → Tempo / Grafana → Tempo	차단(권장)	OTLP 수신은 내부에서만 사용
MinIO	9000(S3), 9001(Console)	Mimir/Tempo ↔ MinIO	차단	S3 호환 스토리지

2. 구축 과정(주차별 수행 내역 및 주요 설정)

2.1 주차별 수행 내역 요약

주차	핵심 목표	주요 수행 항목	산출물/검증
1주차	베이스 스택 기동/연동	인스턴스 생성, docker-compose.yml파일 및 설정파일 작성, 각 컴포넌트 구성, 로그/메트릭 파이프라인 완성	Grafana UI 접속, Loki, mimir, Tempo 데이터소스 연결 확인, 모니터링 알람 설정 등 필수 산출물 git output폴더에 정리
2주차	고도화 + 통합 안정화 + 문서화	컴포넌트 이미지 버전 고정, 보안(least privilege), Grafana provisioning, 로그 샘플링, Alloy 마이그레이션	대시보드/알람 정상 동작, 트러블슈팅 3건 정리, 결과 보고서 작성

2.2 주요설정 설명

2.2.1 Loki 설정 핵심 옵션 분석(`loki-config.yaml`)

다음 설정은 Loki 단일 인스턴스(단일 VM)에서도 운영 품질을 확보하기 위한 최소 구성이며, 특히 **보관 정책(retention)**을 명시하고 **compact**로 이를 실제 적용하는 것이 핵심이다.

- `auth_enabled: false`
 - 의미:** Loki API 접근 인증을 비활성화. 단일 VM 실습/학습 환경에서는 편의성이 높지만, 실제 운영에서는 Reverse Proxy 인증 또는 네트워크 차단이 권장된다.
 - 운영 관점:** 본 프로젝트는 Loki 포트를 외부에 오픈하지 않음(보안그룹 차단)
- `schema_config.store: tsdb + schema: v13 + index.period: 24h`
 - 의미:** Loki의 저장 방식(인덱스/체크)을 TSDB 방식으로 구성. 인덱스 주기를 24시간 단위로 설정하여 관리 단위를 명확히 한다.
 - 효과:** 추후 보관/삭제 정책 적용 시 "일 단위 파티셔닝"이 되어 운영상 예측 가능성이 증가한다.
- `limits_config.retention_period: 168h (7일)`

- **의미:** 로그 보관 기간을 7일로 제한. 무제한 보관 시 디스크가 빠르게 소진될 수 있으므로 상한을 둔다.
- **주의:** Loki에서 retention은 단순히 값 설정만으로 끝나지 않으며, 실제 삭제는 compactor가 수행한다.
- `compactor.retention_enabled: true`
 - **의미:** retention 정책을 실제로 적용(삭제 요청/체크 삭제 등)하도록 compactor를 활성화한다.
 - **핵심 포인트:** `retention_period` 만 설정하면 “정책 선언”에 그칠 수 있으나, compactor가 동작해야 “정책 집행”이 된다.

2.2.2 Promtail 설정 핵심 옵션 분석(`promtail-config.yaml`)

- `positions.filename`
 - **의미:** Promtail이 마지막으로 처리한 파일 offset을 기록해, 컨테이너 재시작 후에도 “중복 수집/누락”을 최소화한다.
 - **운영 관점:** positions가 없으면 재기동 시 과거 로그를 다시 push하거나, 반대로 일부를 놓칠 수 있다.
- `clients.url`
 - **의미:** Promtail이 push할 Loki endpoint.
 - **효과:** Compose 네트워크 내부 DNS(서비스명 `loki`)를 사용하므로, IP 변화와 무관하게 안정적으로 연결된다.
- `__path__: /var/log/*log`
 - **의미:** VM 호스트의 시스템 로그를 Promtail 컨테이너에 읽기 전용(`/var/log:/var/log:ro`) 마운트하고, 해당 경로를 수집 대상으로 지정한다.
 - **보안 포인트:** `:ro` 로 설정하여 수집 에이전트가 호스트 로그를 변경 할 수 없도록 차단했다.

2.2.3 Prometheus 설정 핵심 옵션 분석(`prometheus.yml`)

- `global.scrape_interval: 15s`
 - **의미:** 15초 간격으로 시계열 데이터를 수집.
- `remote_write` (Prometheus → Mimir)
 - **의미:** Prometheus를 “수집기”로 두고, 장기 저장은 Mimir로 위임한다.
 - **효과:** Prometheus 로컬 디스크는 상대적으로 가볍게 유지하면서도, Grafana에서 장기간 메트릭 조회가 가능해진다.

2.2.4 Mimir 설정 핵심 옵션 분석(`mimir-config.yaml`)

- `target: all` (Monolithic mode)
 - **의미:** Mimir를 단일 컨테이너로 구동하여, 분산 구성의 운영 복잡도를 제거한다.
 - **운영 관점:** 추후 Kubernetes 전환 시 microservices mode로 확장 가능하다.
- `multitenancy_enabled: false`
 - **의미:** 멀티 테넌시를 비활성화하여 `X-Scope-OrgID` 같은 org id 헤더가 없어도 동작하도록 한다.
 - **효과:** `401 Unauthorized: no org id` 문제를 예방한다.
- `blocks_storage.backend: s3` + MinIO 연동
 - **의미:** TSDB 블록을 S3 호환 스토리지로 저장한다.
 - **효과:** 디스크 고정 용량의 단일 VM에서도 객체 스토리지 모델을 학습할 수 있다.

2.3 LGTM 옴저버빌리티 스택 고도화 세부 내역

2.3.1 환경 일관성 및 재현 안정성 확보: Image Version Pinning

배경

Docker 이미지 태그 중 `:latest` 는 편리하지만, 다음과 같은 운영 리스크가 있다.

- 재기동/재배포 시점에 **새 이미지가 자동으로 선택**될 수 있어, 동일한 구성 파일로도 동작이 달라진다.
- 메이저 버전 변경이 포함될 경우 **설정 키 변경/기본값 변화/플러그인 호환성 붕괴**가 발생할 수 있다.
- 트러블슈팅 시 “동일 문제 재현”이 어려워지고, 문서화된 환경과 실제 환경의 차이가 커진다.

조치

- `docker-compose.yml` 에서 이미지 태그를 명시적으로 고정했다.
- 실행 중 컨테이너의 바이너리/레지스트리 digest를 대조하여 실제 구동 버전을 식별하고, 문서/코드에 반영했다.

효과

- 누구든 동일 리포지토리를 clone하고 compose up 하면 **동일 버전**으로 재현 가능
- 장애 발생 시 버전에 대한 변수를 제거하여 원인 분석이 단순해짐
- 결과보고서 및 README의 신뢰도가 상승

2.3.2 Grafana Provisioning 자동화: DataSource/Dashboard/Alert를 코드로 관리

배경

Grafana는 UI 기반으로 빠르게 대시보드/알림을 만들 수 있지만, 운영 환경에서는 다음 문제가 발생한다.

- UI에서만 설정이 존재하면 “누가/언제/왜 변경했는지” 추적이 어려움
- 다른 환경(개발/스테이징/운영)으로 이관 시 수동 클릭이 반복되어 오류 가능성 증가
- 장애 복구(재배포) 시 초기 상태를 빠르게 복원하기 어려움

조치

Grafana의 provisioning 디렉터리를 컨테이너에 마운트하여 다음 자산을 코드로 관리하도록 구성했다.

- `provisioning/datasources` : Loki/Mimir/Tempo 데이터소스 자동 등록
- `provisioning/dashboards` : JSON 대시보드 자동 로드
- `provisioning/alerting` : Alert rule 자산화(파일 기반)

또한 환경 변수 기반 기능(`GF_PROVISIONING_ENV_VARS_ENABLE=true`)을 활용하여, 민감 정보(예: SMTP 계정)는 `.env` 에서 주입하고 코드에는 노출되지 않도록 분리했다.

효과

- 재배포 시 **클릭 없이 동일한 대시보드/알림/데이터소스가 자동 복원**
- GitHub 커밋 히스토리로 변경 추적 가능
- 구성의 표준화로 신뢰성, 재현성 확보
- 구성의 표준화로 신뢰성, 재현성 확보

2.3.3 Tempo 컨테이너 Root 권한 제거: Init 컨테이너 기법(Least Privilege)

배경

Tempo는 WAL 및 설정 파일 접근을 위해 특정 경로(`/var/tempo/wal`, `/etc/tempo`)에 읽기/쓰기 권한이 필요하다. 단일 컨테이너에서 이를 단순히 해결하려고 `root` 로 실행하면 다음 문제가 발생한다.

- 컨테이너 내부 프로세스가 root 권한을 가지므로, 취약점 악용 시 피해 범위가 커진다.
- 호스트 마운트/볼륨이 있을 경우 권한 오남용 가능성이 증가한다.

조치

- `tempo-init` 컨테이너를 추가하여, 시작 시 필요한 디렉터리의 소유권을 Tempo 컨테이너 사용자 UID/GID `10001` 로 변경한 후 종료되게 했다.
- 이후 `tempo` 본 컨테이너는 `root` 로 실행하지 않도록 권한을 제거했다.

효과

- **최소 권한 원칙(Least Privilege)** 준수로 보안 리스크 감소
- 권한 문제로 컨테이너가 즉시 종료되는 장애의 재발 방지
- “Init 컨테이너로 선작업 → 본 서비스는 최소 권한”이라는 운영 패턴을 학습/적용

2.3.4 로그 레벨 기반 샘플링(Noise Reduction): INFO 로그 Drop

배경

운영/실습 환경에서 애플리케이션 및 시스템 로그는 `INFO` 레벨의 정상 동작 로그가 대다수를 차지한다. 이를 모두 수집/적재하면 다음과 같은 문제가 발생한다.

- Loki 저장소/디스크 사용량이 빠르게 증가하여 **retention 기간 내에도 용량 압박**이 생길 수 있다.
- Grafana에서 LogQL 조회 시 **불필요한 노이즈**가 증가해, 장애/이상 징후(예: ERROR/WARN) 탐지가 느려진다.
- 단일 VM 환경에서는 IO/CPU 자원이 제한적이므로, 로그 수집/전송 비용이 커질수록 전체 스택 안정성에 영향을 줄 수 있다.

따라서 “항상 필요한 로그(ERROR/WARN)는 남기되, 상대적으로 가치가 낮은 INFO 로그는 샘플링/필터링”하여 비용 대비 효율을 높이는 전략이 필요하다.

조치

Promtail의 `pipeline_stages` 를 사용해 로그 본문에서 JSON 필드의 `"level": "INFO"` 값을 정규식으로 추출한 뒤, `INFO` 로그만 drop 하도록 구성하였다.

효과

- INFO 로그 적재량이 감소하여 **저장 비용 및 디스크 사용량을 절감**하고 retention 정책 운영이 용이해진다.
- LogQL 조회 시 노이즈가 줄어 **WARN/ERROR 중심의 트러블슈팅 속도**가 향상된다.

2.3.5 Promtail, Prometheus → Grafana Alloy 마이그레이션 테스트

배경

- Grafana Labs는 Promtail 사용자에게 **Grafana Alloy**로의 마이그레이션을 권장하고 있다.
- 또한 향후 쿠버네티스 기반 환경으로 전환할 경우, 메트릭, 트레이스, 로그 수집을 **단일 에이전트로 통합**하는 방향이 운영 측면에서 더 유리하다고 판단하여 사전 검증을 수행했다.

조치

- **운영 안정성 보장:** 기존 `main` 브랜치는 그대로 유지하고, 별도의 `alloy` 브랜치를 생성하여 변경을 격리했다.
- **검증 환경 분리:** 신규 VM 인스턴스를 추가로 생성하여(접속 IP: `1.201.177.162`) 테스트 환경을 완전히 분리한 뒤, 해당 인스턴스에서만 마이그레이션 작업을 진행했다.
- **마이그레이션 수행 및 성공:** `Promtail` + `Prometheus` 기반 수집 구성을 제거하고, **Grafana Alloy 단일 프로세스로** 메트릭/로그 수집 파이프라인을 구성하여 마이그레이션을 완료했다.
- **검증 채널**
 - Alloy UI: <http://1.201.177.162:12345/>
 - Grafana UI: <http://1.201.177.162:3000>

효과 및 의의

- **에이전트 운영 단순화(구성 요소 축소):** 기존에는 로그(`Promtail`)와 메트릭(`Prometheus`)을 각각 운영해야 했으나, Alloy로 통합하면 **배포/업데이트/롤백/버전 관리 대상이 1개로 축소**되어 운영 부담이 감소한다.
- **수집 파이프라인의 선언형 일원화:** 메트릭 스크래핑과 로그 파이프라인(필터링/레이블링/라우팅)을 한 곳에서 정의·관리할 수 있어, 변경 시점에 발생하기 쉬운 **설정 불일치(스크랩 타겟/레이블 규칙/필터 규칙 분리)** 리스크를 낮출 수 있다.
- **K8s 전환 시 ConfigMap 관리 단순화:** 클러스터 환경에서는 노드 단위로 수집기를 배포하게 되는데, Alloy 기반으로 통합하면 **리소스/권한(RBAC)/ConfigMap/배포 정책**을 단일 수집기로 표준화할 수 있어 운영 효율성이 향상된다.
- **디버깅 가시성 향상(Alloy UI 기반):** Alloy 내장 UI를 통해 컴포넌트 간 데이터 흐름을 확인할 수 있어, 장애 발생 시 “백엔드 문제”인지 “수집단 문제(스크랩 실패/테일 실패/라우팅 오류)”인지의 구분이 빨라져 트러블슈팅 시간을 단축할 수 있다.

3. 검증 결과

3.1 서비스 정상 구동 상태(`docker compose ps`)

프로젝트 요구사항상 “전체 서비스가 정상 구동(Up) 상태”임을 확인해야 한다. 본 프로젝트는 다음 서비스 구성을 기반으로 운영되며, Compose 상 서비스 목록은 아래와 같다.

```
ubuntu@vm-project1-1g7m:~/LGTm_Observerbility_stack$ docker compose ps
```

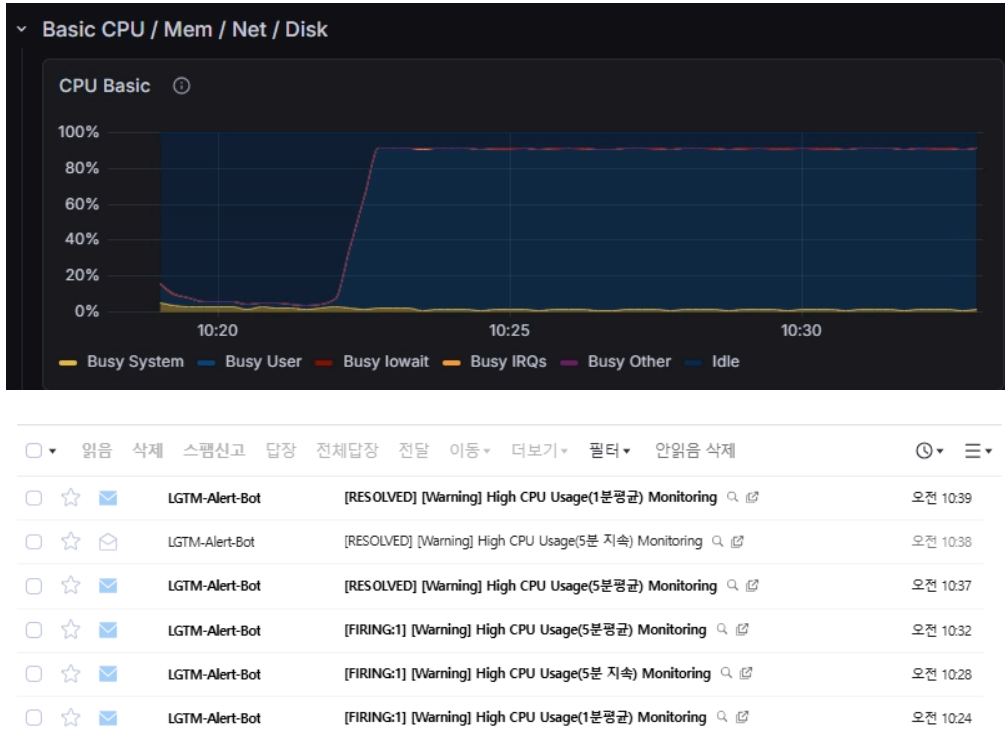
NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
grafana	grafana/grafana:13.0	"/run.sh"	grafana	4 days ago	Up 4 days
loki	grafana/loki:3.7.2	"/usr/bin/loki -conf..."	loki	4 days ago	Up 4 days
mimir	grafana/mimir:3.1.0	"/bin/mimir -config..."	mimir	4 days ago	Up 4 days
minio	minio/minio	"/usr/bin/docker-ent..."	minio	4 days ago	Up 4 days (healthy)
node-exporter	prom/node-exporter:v1.11.1	"/bin/node_exporter ..."	node-exporter	4 days ago	Up 4 days
prometheus	prom/prometheus:v3.12.0	"/bin/prometheus --c..."	prometheus	4 days ago	Up 4 days
promtail	grafana/promtail:3.6.8	"/usr/bin/promtail -..."	promtail	4 days ago	Up 4 days
tempo	grafana/tempo:3.0.0	"/tempo -config.file..."	tempo	4 days ago	Up 4 days

3.2 장애 알람 시나리오

3.2.1 CPU 부하 테스트

- **알람 조건:**
 - CPU Usage > 80% 5분 지속 시,
 - CPU Usage 5분 평균값 > 80% 5분 지속 시
 - CPU Usage 1분 평균값 > 80% 1분 지속 시

```
ubuntu@vm-project1-lgtm:~/LGTMonitorability_stack$ stress-ng --cpu 4 --cpu-load 90 --timeout 720s
stress-ng: info: [2647189] setting to a 720 second (12 mins, 0.00 secs) run per stressor
stress-ng: info: [2647189] dispatching hogs: 4 cpu
```

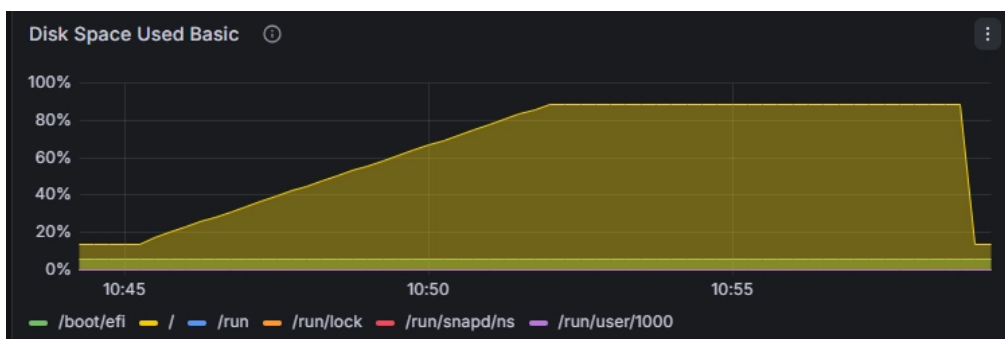


3.2.1 DISK 부하 테스트

• 알람 조건

- DISK Usage > 85% 수치 상회 즉시
- DISK Usage >85% 5분 지속 시

```
ubuntu@vm-project1-lgtm:~/LGTMonitorability_stack$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1       49G   6.6G   42G  14% /
ubuntu@vm-project1-lgtm:~/LGTMonitorability_stack$ sudo dd if=/dev/zero of=/tmp/disk_test_
file bs=1G count=36 status=progress
38654705664 bytes (39 GB, 36 GiB) copied, 400 s, 96.6 MB/s
36+0 records in
36+0 records out
38654705664 bytes (39 GB, 36 GiB) copied, 400.294 s, 96.6 MB/s
ubuntu@vm-project1-lgtm:~/LGTMonitorability_stack$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1       49G   43G   5.7G  89% /
```



			LGTM-Alert-Bot	[RESOLVED] [Critical] High Disk Usage(즉시 알림) Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 11:02
			LGTM-Alert-Bot	[RESOLVED] [Warning] High Disk Usage Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 11:02
			LGTM-Alert-Bot	[FIRING:1] [Warning] High Disk Usage Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 10:57
			LGTM-Alert-Bot	[FIRING:1] [Critical] High Disk Usage(즉시 알림) Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 10:52

4. 트러블슈팅

4.1 이슈 1: MinIO 준비 전 종속 컨테이너 초기화 실패

1) 현상

- `create-bucket`, `mimir`, `tempo` 컨테이너가 오작동.
- 스택 전체 기동이 불안정해지고 “재시작 반복” 또는 “버킷 생성 실패”가 반복됨.

2) 원인(Root Cause)

- Docker Compose는 기본적으로 컨테이너 시작 순서만 제어할 뿐, 서비스 준비 완료를 보장하지 않는다.
- MinIO는 프로세스 시작 이후에도 내부 초기화 시간이 필요하며, 그 사이 종속 서비스가 S3 endpoint를 호출하면 실패한다.

3) 해결(Resolution)

- MinIO에 `/minio/health/live` 기반 healthcheck를 추가하여 “Ready”를 판별
- `depends_on`에 `condition: service_healthy`를 적용하여 MinIO가 healthy 상태가 된 후에만 종속 컨테이너가 실행되도록 통제

4) 결과(Outcome)

- MinIO 준비 완료 이후에b만 종속 서비스가 시작되어 초기화 실패가 제거됨
- 버킷 생성 및 S3 연동 안정성이 확보되어 장기 운용 시 재현 가능성이 향상됨

4.2 이슈 2: Grafana 대시보드 프로비저닝 실패

1) 현상(Symptom)

- Grafana 파일 기반 대시보드 프로비저닝(dashboards provisioning) 적용 시, JSON 파일이 “유효하지 않은 파일”로 판단되어 자동 로드가 차단됨
- UI에서는 대시보드가 보이지 않음

2) 원인(Root Cause)

- Grafana UI에서 Export한 대시보드가 웹사이트 Import용인 `V2 Resource`로 추출됨
- 파일 프로비저닝 엔진은 `V2 Resource` 스키마 구조를 기대하지 않아 JSON을 정상 파싱하지 못한다.

3) 해결(Resolution)

- Grafana 대시보드 Export 시 **Advanced options**로 진입하여
 - Model을 `Classic`으로 전환
 - `Classic` 모드에서 **Raw JSON** 형태로 Export
- Export한 Classic Raw JSON을 provisioning 대상 경로에 배치 후 Grafana 재기동

4) 결과(Outcome)

- 수동 클릭/우회 API 없이, 컨테이너 기동 시점에 대시보드가 자동으로 로드되는 것을 확인
- 환경 재현성이 향상됨

4.3 이슈 3: Grafana 대시보드 연결 실패

1) 현상(Symptom)

- 데이터 소스(Data Source) 설정 화면에서 'Save & Test' 실행 시 정상(Success)으로 연동됨을 확인.
- 하지만 실제 메트릭 데이터 대시보드상에는 데이터가 시각화되지 않거나, 연결이 끊어진 것처럼 빈 화면(No Data)이 노출됨.

2) 원인(Root Cause)

- 과거에 사용하던 기존 Job 이름들이 데이터 소스나 수집기 측에 여전히 다수 남아 있는 상태 확인.
- Save & Test는 데이터 소스 엔진 자체의 통신상태만 검증하기 때문에 정상으로 통과했으나, 실제 대시보드 패널들은 변경된 최신 Job 이름을 바라보지 못하고 과거의 Job이름으로 메트릭을 조회함.

3) 해결(Resolution)

- Job 이름 정제 및 매핑 수정
 - 대시보드 환경설정 및 쿼리문을 점검하여 과거 이름으로 박혀있던 레거시 Job명칭을 업데이트함
- 불필요한 레거시 메트릭 정리

4) 결과(Outcome)

- 대시보드 패널과 실제 수집되는 메트릭 Job이름이 일치되면서, 대시보드 상에 데이터가 정상적으로 실시간 시각화 됨을 확인

5. 회고 및 개선 제안

5.1 배운 점

- Logs/Metrics/Traces는 각각 단독으로도 가치가 있지만, MSA 환경에서의 문제 해결 속도는 **3개 신호를 상호 연계**할 때 비약적으로 개선된다.
 - 예: CPU 급증(메트릭) 시점의 오류 로그(로그), 특정 요청의 병목 구간(트레이스)까지 한 번에 연결해 추적 가능
- Docker Compose 기반 단일 VM 실습이라도, **의존성(healthcheck/depends_on)**, **권한(least privilege)**, **구성 자동화(provisioning)**를 적용하면 품질을 크게 끌어올릴 수 있음을 느꼈다.
- 이 프로젝트에서는 **:latest 태그**를 붙여서 작업을 하고, 후에 **버전정보**를 추출하여 고정하는 방식을 사용하였으나 다음부터는 **각 기술스택의 버전정보**를 미리 확인하고 고정한 후 구축해야 품질을 더 높일 수 있음을 느꼈다.

5.2 한계점

- 단일 VM + Docker Compose 환경이므로, 실제 MSA 환경처럼 컴포넌트를 분산 배치하거나 HA/Replica 기반 확장을 실험하기 어렵다.
- 애플리케이션 연동이 제한적이어서, 트레이스는 인프라 측면에서의 기반 구축 위주로 검증되었다.

5.3 다음 단계

- **Kubernetes(K8s) 전환:** Docker Compose → K8s로 이관하고 Helm 기반(loki-distributed, mimir-distributed, tempo-distributed)으로 분산화를 구현
- **컴포넌트 분산 :** Distributor, Ingestor, Compactor등의 컴포넌트들을 분산하고 세팅하여 MSA화
- **오토스케일링/스토리지 확장:** HPA/VPA, object storage 확장, retention/compaction 정책을 실제 운영 수준으로 고도화
- **실제 애플리케이션 연동:** Spring Boot/Node.js 서비스에 OpenTelemetry SDK를 적용하여 “요청 단위 장애 분석” 트레이싱을 실현 수준으로 강화

6. 종합 및 결론

본 프로젝트는 단일 Cloud VM 환경 내에서 LGTM(Loki, Grafana, Tempo, Mimir) 기반의 통합 관찰성 파이프라인을 성공적으로 구축하고 고도화하였습니다.

- **관찰성 통합 체계 마련:** 분산되어 있던 로그, 메트릭, 트레이스를 Grafana 단일 뷰포트로 통합함으로써 시스템 이상 징후 감지부터 원인 분석까지의 과정을 단일 워크플로우로 연계할 수 있는 기반을 다졌습니다.
- **엔지니어링 완성도 확보:** 단순 기능 구현에 그치지 않고, 이미지 버전 고정(Pinning), Grafana 프로비저닝 자동화, 인프라 보안 권한 최적화(Least Privilege), Promtail의 로그 샘플링 정책 등을 적용하여 실제 운영 환경에 준하는 재현성과 안정성을 확보하였습니다.
- **미래 아키텍처 다변화 검증:** 2주차에 진행된 Grafana Alloy로의 마이그레이션 테스트를 통해 수집 에이전트의 일원화 가능성을 확인하였으며, 이는 향후 쿠버네티스(K8s) 클러스터 환경으로의 확장 및 모니터링 컴포넌트 관리를 효율화하는 최적의 디딤돌이 될 것입니다.

본 프로젝트를 통해 확보한 오픈소스 기반의 모니터링 설계 역량과 트러블슈팅 경험은 향후 대규모 MSA 환경으로의 인프라 확장 및 중단 없는 서비스 운영을 위한 견고한 기술적 자산이 될 것으로 기대됩니다.

[이상 보고서 끝]